FATEC BS Rubens Lara

Curso de Ciência de Dados

Disciplina: Masoquismo Funcional

Soma Simbólica Formal em Haskell

Uma Formalização de Máquina de Mealy
para Adição Decimal com Tabela Simbólica Pura

Autor

Peterson Carara Junior

Professor responsável

João Paulo

Santos – SP

23 de setembro de 2026

Resumo

Este documento descreve a formalização, em Haskell, de uma Máquina de Estados Finitos (FSM) de Mealy para a soma simbólica de números decimais representados como `String`. O princípio central do projeto é que a operação permaneça *inteiramente no domínio dos caracteres*, sem qualquer conversão para tipos numéricos nativos como `Int` ou `Integer`, e sem mesmo introduzir um tipo intermediário de dígitos: as transições são definidas por uma *tabela simbólica explícita* sobre pares de `Char`. O documento apresenta a arquitetura da solução, o código comentado, o diagrama da FSM e um glossário detalhado dos termos técnicos empregados.

Palavras-chave: Programação Funcional. Haskell. Máquina de Mealy. Tabela Simbólica. Soma Decimal. Pureza Referencial.

Sumário

1 Introdução **3**
1.1 Princípio da Pureza Simbólica 3

2 Fundamentação Teórica **3**
2.1 Definição Formal da FSM de Mealy 3
2.2 A Tabela Simbólica como Tabela de Transição 4
2.3 Comutatividade e Redução da Tabela 4

3 Diagrama da FSM **4**
3.1 Leitura do Diagrama 5

4 Arquitetura do Código **6**

5 Código Comentado **6**
5.1 Tabela Simbólica de Transição 6
5.2 Auxiliar Recursivo 6
5.3 Soma de Duas Strings 7
5.4 Soma de Lista 7

6 Propriedades e Análise **7**
6.1 Completude Funcional 7
6.2 Segurança de Tipos 7
6.3 Totalidade Funcional 7
6.4 Complexidade 7
6.5 Pureza Absoluta 8

7 Glossário de Termos **8**
7.1 Termos de Haskell 8
7.2 Termos Específicos do Projeto 8

8 Conclusão **9**

1 Introdução

O objetivo deste projeto é formalizar, em Haskell, o algoritmo clássico de soma decimal posição-a-posição, tratando-o como uma **Máquina de Mealy** cujo estado é o transporte (*vai-um*) e cuja entrada é o par de caracteres na mesma posição.

A escolha de Haskell é sustentada por três propriedades da linguagem:

1. **Pattern matching exaustivo** sobre os 100 pares de caracteres possíveis, garantindo completude funcional em tempo de compilação.
2. **Type safety**: a inferência de tipos precede a avaliação, de modo que erros de tipo nunca ocorrem em tempo de execução [1].
3. **Funções puras**: cada transição é uma equação determinística, sem efeitos colaterais.

1.1 Princípio da Pureza Simbólica

Um dos princípios norteadores deste projeto é que a operação de soma permaneça **inteiramente no domínio dos caracteres**. Diferentemente de outras formalizações que introduzem um tipo intermediário de dígitos, esta implementação opera *diretamente* sobre `Char`.

Formalmente, seja $\mathcal{A} = \{\texttt{'0'}, \texttt{'1'}, \ldots, \texttt{'9'}\}$ o alfabeto efetivo dos caracteres utilizados. A função de transição

$$\delta : \mathcal{A} \times \mathcal{A} \times \mathcal{V} \to \mathcal{A} \times \mathcal{V},$$

onde $\mathcal{V} = \{\texttt{'0'}, \texttt{'1'}\}$ é o alfabeto do estado de transporte, é definida como uma **tabela explícita** de regras simbólicas. Cada linha da definição de `sd` corresponde a uma regra $(c_1, c_2, v) \mapsto (r, v')$, onde r é o caractere resultante e v' o novo estado de transporte.

Essa escolha tem três consequências importantes:

1. **Pureza referencial absoluta**: não há dependência de tipos numéricos nativos, nem mesmo de um tipo intermediário de dígitos.
2. **Transparência estrutural**: a função de transição é literalmente uma tabela finita de 100 casos (reduzida a 55 com a regra de comutatividade final), diretamente verificável por inspeção.
3. **Compatibilidade com o modelo formal**: a implementação espelha exatamente a definição matemática de Máquina de Mealy.

2 Fundamentação Teórica

2.1 Definição Formal da FSM de Mealy

Uma Máquina de Mealy é uma 6-tupla $\mathcal{M} = (Q, \Sigma, \Lambda, \delta, \lambda, q_0)$, onde Q é o conjunto finito de estados, Σ o alfabeto de entrada, Λ o alfabeto de saída, $\delta : Q \times \Sigma \to Q$ a função de transição, $\lambda : Q \times \Sigma \to \Lambda$ a função de saída e $q_0 \in Q$ o estado inicial.
Para a soma decimal implementada:

$$\begin{aligned} Q &= \{\texttt{'0'}, \texttt{'1'}\}, \\ \Sigma &= \texttt{Char} \times \texttt{Char}, \\ \Lambda &= \texttt{Char}, \\ q_0 &= \texttt{'0'}. \end{aligned}$$

2.2 A Tabela Simbólica como Tabela de Transição

A função `sd` (*soma de dígitos*) define explicitamente a função de transição combinada δ e a função de saída λ em uma única tabela. Cada equação é uma tupla $(c_1, c_2, v) \mapsto (r, v')$, representando a computação:

$$\lambda(c_1, c_2, v) = r, \qquad \delta(c_1, c_2, v) = v'.$$

O uso de `Char` diretamente (em vez de um tipo `Digito`) torna a tabela o mais próximo possível do modelo matemático abstrato, sem camadas intermediárias de abstração.

2.3 Comutatividade e Redução da Tabela

A equação final

```
sd a b v = sd b a v
```

implementa a **comutatividade da soma**, cobrindo automaticamente os casos em que o primeiro argumento é maior que o segundo. Isso reduz o número de equações explícitas de 100 para 55.

3 Diagrama da FSM

A Figura 1 apresenta o diagrama de estados da Máquina de Mealy para a soma decimal implementada. O estado inicial é `SemVaiUm` (transporte `'0'`); o estado `ComVaiUm` (transporte `'1'`) indica que há um transporte pendente para a próxima posição. As quatro transições cobrem todos os casos possíveis da tabela simbólica `sd`.

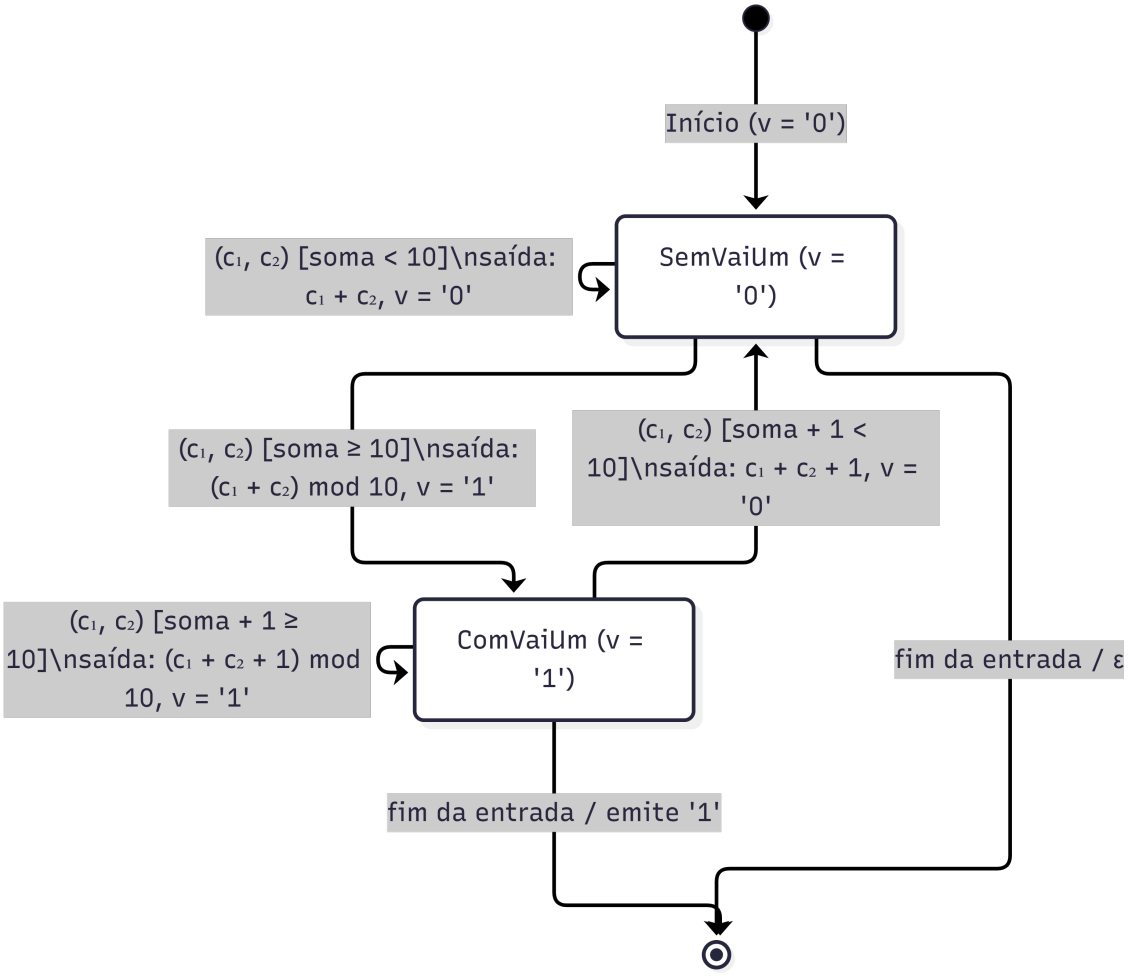


Figura 1: Diagrama de estados da Máquina de Mealy para a soma decimal. O estado inicial é `SemVaiUm` (transporte `'0'`); o estado final é representado pelo círculo duplo. As quatro transições cobrem todos os casos possíveis da tabela simbólica `sd`, com a comutatividade tratada pela equação final `sd a b v = sd b a v`.

3.1 Leitura do Diagrama

As quatro transições do autômato podem ser descritas da seguinte forma:

- **(1)** Soma simples sem transporte, resultado < 10: permanece em `SemVaiUm`, emitindo $c_1 + c_2$.
- **(2)** Soma simples com transporte, resultado ≥ 10: transita para `ComVaiUm`, emitindo $(c_1 + c_2) \bmod 10$.
- **(3)** Soma com transporte de entrada, resultado $+1 < 10$: retorna a `SemVaiUm`, emitindo $c_1 + c_2 + 1$.
- **(4)** Soma com transporte de entrada, resultado $+1 \geq 10$: permanece em `ComVaiUm`, emitindo $(c_1 + c_2 + 1) \bmod 10$.

Ao final da entrada, se o estado for `ComVaiUm`, emite-se o caractere `'1'` adicional; caso contrário, emite-se a palavra vazia ε. Na implementação, isso corresponde aos casos base `sa [] [] '1' = ['1']` e `sa [] [] '0' = []`.

4 Arquitetura do Código

O módulo `SomaSimbolica` é organizado em quatro seções claras:

1. **Tabela simbólica** (`sd`): função de transição explícita sobre pares de `Char`.
2. **Auxiliar recursivo** (`sa`): percorre duas `String` alinhadas da direita para a esquerda, propagando o estado de transporte.
3. **Soma de duas entradas** (`s`): inverte as entradas, invoca `sa`, e inverte o resultado.
4. **Soma de lista** (`sl`): dobra a operação sobre uma lista de `String` via `foldl1`.

5 Código Comentado

5.1 Tabela Simbólica de Transição

```
sd :: Char -> Char -> Char -> (Char, Char)
sd '0' '0' '0' = ('0', '0')
sd '0' '0' '1' = ('1', '0')
sd '0' '1' '0' = ('1', '0')
sd '0' '1' '1' = ('2', '0')
-- ... (demais casos) ...
sd '9' '9' '0' = ('8', '1')
sd '9' '9' '1' = ('9', '1')

-- Comutatividade: cobre casos em que a > b.
sd a b v = sd b a v
```

Listing 1: Tabela simbólica `sd` (trecho).

Cada linha da tabela é uma **regra simbólica atômica**: dada a tríade (c_1, c_2, v), produz o par (r, v'). A tabela cobre todos os casos base possíveis para $c_1 \leq c_2$, sendo completada pela equação final de comutatividade.

5.2 Auxiliar Recursivo

```
sa :: String -> String -> Char -> String
sa [] [] '0' = []
sa [] [] '1' = ['1']
sa (x:xs) [] v = let (r, nv) = sd x '0' v in r : sa xs [] nv
sa [] (y:ys) v = let (r, nv) = sd '0' y v in r : sa [] ys nv
sa (x:xs) (y:ys) v = let (r, nv) = sd x y v in r : sa xs ys nv
```

Listing 2: Função auxiliar recursiva `sa`.

A função `sa` (*soma auxiliar*) percorre as duas listas de caracteres simultaneamente, aplicando a tabela `sd` em cada posição e propagando o novo estado. Os cinco casos cobrem:

- **Caso base 1** ([], [], $'0'$): nenhum transporte residual; resultado vazio.
- **Caso base 2** ([], [], $'1'$): transporte final pendente; emite o caractere `'1'`.
- **Caso 3** (primeira lista não-vazia, segunda vazia): alinha a segunda com `'0'`.
- **Caso 4** (primeira vazia, segunda não-vazia): alinha a primeira com `'0'`.
- **Caso 5** (ambas não-vazias): aplica `sd` aos primeiros elementos e recursa.

5.3 Soma de Duas Strings

```
s :: String -> String -> String
s a b = reverse (sa (reverse a) (reverse b) '0')
```

Listing 3: Função `s` de soma de duas strings.

A função `s` inverte ambas as entradas (para alinhá-las pela direita, tratando primeiro as unidades), invoca `sa` com o estado inicial `'0'`, e inverte o resultado. Essa estratégia reflete a maneira como humanos somam números decimais manualmente.

5.4 Soma de Lista

```
sl :: [String] -> String
sl [] = "0"
sl xs = foldl1 s xs
```

Listing 4: Função `sl` de soma de lista.

A função `sl` generaliza `s` para listas de strings via `foldl1`, que reduz a lista aplicando `s` acumulativamente. O caso base `"0"` trata a lista vazia.

6 Propriedades e Análise

6.1 Completude Funcional

A exaustividade do pattern matching em `sd` sobre os 100 pares de `Char` garante que **nenhuma combinação de dígitos** deixa de ser tratada. Como a equação final cobre os casos restantes por comutatividade, o compilador GHC não emite aviso de padrões incompletos [1].

6.2 Segurança de Tipos

Segundo Hutton [1], "a inferência de tipos precede a avaliação", de modo que "erros de tipo nunca podem ocorrer durante a avaliação". Neste projeto, isso se manifesta em:

1. Impossibilidade de passar algo que não seja `Char` para `sd`.
2. Impossibilidade de compor `s` ou `sl` com funções de tipos incompatíveis.
3. Impossibilidade de construir uma `String` que não seja uma lista de `Char`.

6.3 Totalidade Funcional

Todas as funções do módulo são **total** para entradas válidas: retornam uma `String` bem definida para qualquer combinação de dígitos. Não há uso de `error`, `undefined` ou exceções. Comportamentos indefinidos só ocorreriam se caracteres não-numéricos fossem passados a `sd` — situação não coberta pela tabela.

6.4 Complexidade

Para duas strings de comprimento n, a função `s` executa $O(n)$ chamadas recursivas de `sa`, cada uma realizando uma consulta $O(1)$ à tabela `sd`. A complexidade total é, portanto, $O(n)$, linear no comprimento da entrada.

6.5 Pureza Absoluta

Um aspecto crucial deste projeto é que **em nenhum ponto** há conversão para tipos numéricos. Nem `Int`, nem `Integer`, nem qualquer tipo intermediário: a computação inteira se dá sobre `Char`, preservando o caráter estritamente simbólico da operação.

7 Glossário de Termos

7.1 Termos de Haskell

`Char`
Tipo de caracteres Unicode. Valores literais entre aspas simples: `'0'`, `'a'`, `'\n'`.

`String`
Sinônimo para `[Char]`: lista de caracteres.

Pattern Matching
Mecanismo que compara um valor contra padrões sintáticos para decidir qual equação aplicar.

Pattern Matching Exaustivo
Quando todas as possibilidades são cobertas, garantindo completude funcional.

Recursão
Técnica em que uma função se invoca a si mesma, reduzindo o problema a instâncias menores até atingir um caso base.

`let ... in`
Definição local dentro de uma expressão.

`where`
Cláusula para definições locais.

`reverse`
Inverte uma lista.

`foldl1`
Redução à esquerda sobre lista não-vazia.

Função Pura
Função que sempre produz o mesmo resultado para o mesmo argumento e não tem efeitos colaterais.

Função Total
Função definida para todos os valores de seu domínio.

Type Safety
Erros de tipo nunca ocorrem em tempo de execução.

Type Inference
O compilador deduz os tipos das expressões automaticamente.

7.2 Termos Específicos do Projeto

`sd`
Tabela simbólica de soma de dígitos. Recebe dois `Char` (os dígitos a somar) e um `Char` (o transporte de entrada) e devolve um par `(Char, Char)` com o dígito resultante e o novo transporte.

`sa`
: Função auxiliar recursiva que percorre duas `String` alinhadas, aplicando `sd` em cada posição.

`s` Soma de duas `String`. Inverte as entradas, invoca `sa` e inverte o resultado.

`sl`
: Soma uma lista de `String` via `foldl1`.

Vai-Um (Carry)
: Transporte propagado de uma posição à seguinte quando a soma excede 9. Representado aqui pelo caractere `'1'`.

Pureza Simbólica
: Princípio segundo o qual a operação permanece inteiramente no domínio dos símbolos finitos, sem conversões para tipos numéricos.

8 Conclusão

O projeto demonstra como uma Máquina de Mealy para soma decimal pode ser formalizada em Haskell de forma *minimalista e estritamente simbólica*:

- **Tabela explícita** (`sd`) como função de transição, diretamente verificável por inspeção.
- **Recursão estrutural** (`sa`) que espelha o procedimento humano de soma manual.
- **Composição funcional** (`s`, `sl`) que generaliza para listas de números.
- **Ausência total de conversões numéricas**: nenhum uso de `Int`, `Integer` ou qualquer tipo numérico intermediário.

A pureza referencial do código e o rigor do sistema de tipos de Haskell garantem que o programa seja *estruturalmente correto* por construção, e que todas as funções sejam totais para entradas válidas.

Referências

[1] G. Hutton. *Programming in Haskell*. Cambridge University Press, 2nd edition, 2016.

[2] B. Milewski. *Category Theory for Programmers*. Compilado e editado por I. Tabachnik, 2020.

[3] P. Wadler. Comprehending Monads. *Mathematical Structures in Computer Science*, 2(4):461–493, 1992.

[4] A. G. de Oliveira. *Haskell: Uma Introdução à Programação Funcional*. Casa do Código, 2017.